Technical Design Document

AI Changelog & Release Notes Generator — Architecture, Diagrams & Build Steps

Owner: Srikanth Pusapati · Version 1.0 · July 31, 2026

1. Design Principles

- **Deterministic shell, probabilistic core.** Everything around the LLM call (collection, classification rules, rendering, git operations) is plain deterministic code. The LLM does exactly one job: rewrite grouped facts into audience-appropriate prose, returning JSON that is schema-validated.
- **Human in the loop.** Output lands as a PR or draft release, never an unreviewed publish.
- **Metadata only.** Commit messages, PR titles/bodies and labels go to the LLM — never source diffs by default.
- **Portable core.** The generator is a plain Python package; CLI, GitHub Action, and future SaaS are thin adapters around it.

2. High-Level Architecture (Component View)

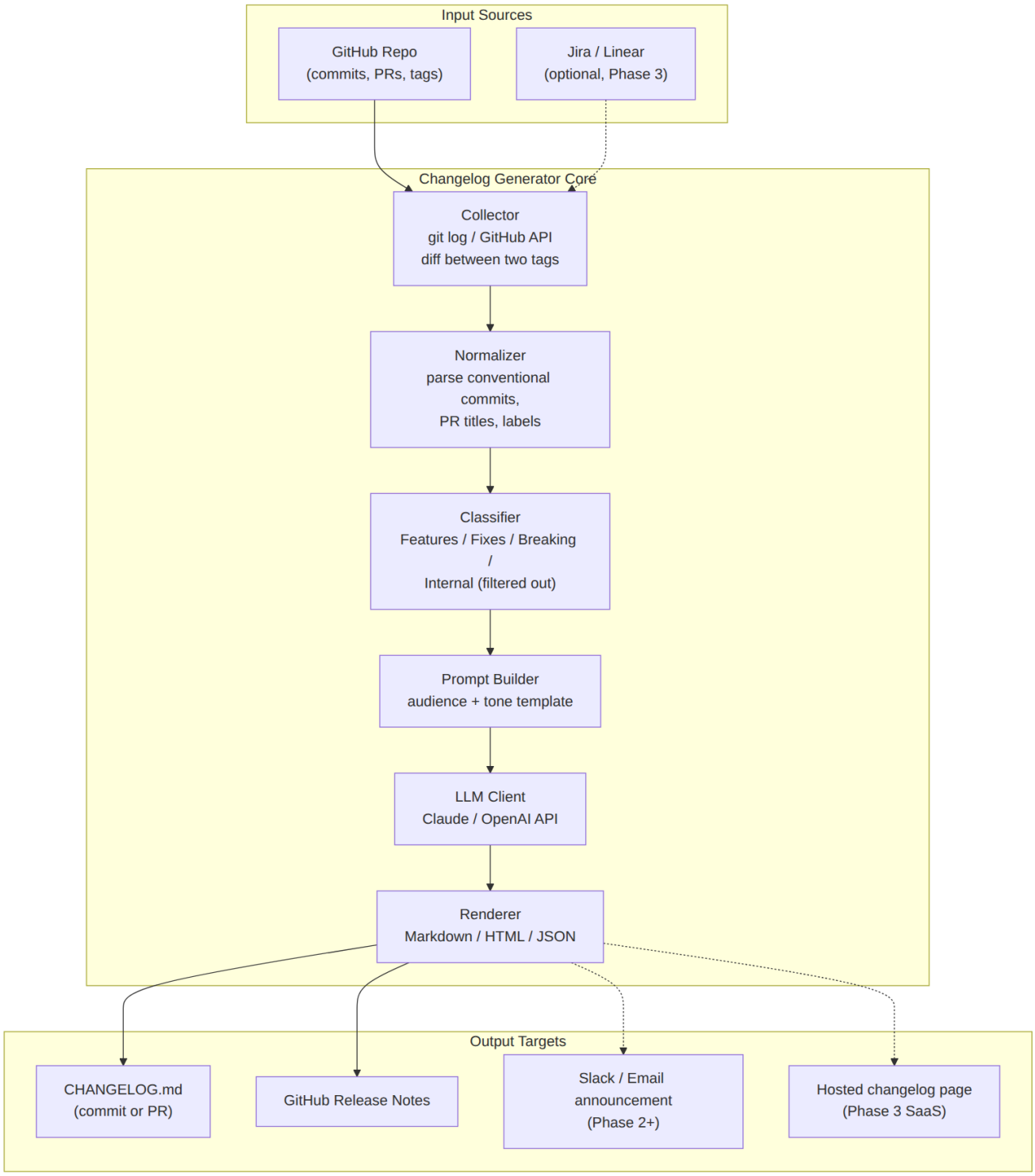

Figure 1 — Component diagram: sources → pipeline → outputs. Dashed edges are later phases.

Component responsibilities

Component	Responsibility	Implementation
Collector	Fetch commits and merged PRs between two refs (tags, SHAs, or dates)	git CLI via subprocess + GitHub REST API (requests); paginate; cache to JSON
Normalizer	Unify commits/PRs into one ChangeItem model; parse Conventional Commit prefixes, PR labels, linked issues	Pure Python, dataclasses; unit-testable
Classifier	Bucket into Features / Improvements / Fixes / Breaking; drop noise (chore, ci, deps, test)	Rules first (prefix/label maps); LLM assist only for unlabeled leftovers
Prompt Builder	Assemble system + user prompt: grouped changes, tone, audience, product context, few-shot examples	Jinja2 templates versioned in-repo
LLM Client	One API call per release; JSON-schema-constrained output; 1 retry on validation failure	Anthropic/OpenAI SDK behind a small interface
Renderer	Markdown (Keep-a-Changelog), GitHub Release body, JSON	Jinja2 templates; idempotent
Output Adapters	Write file, open PR, publish release; later Slack/email/pages	GitHub API; Action uses GITHUB_TOKEN

3. Generation Flow (Sequence View)

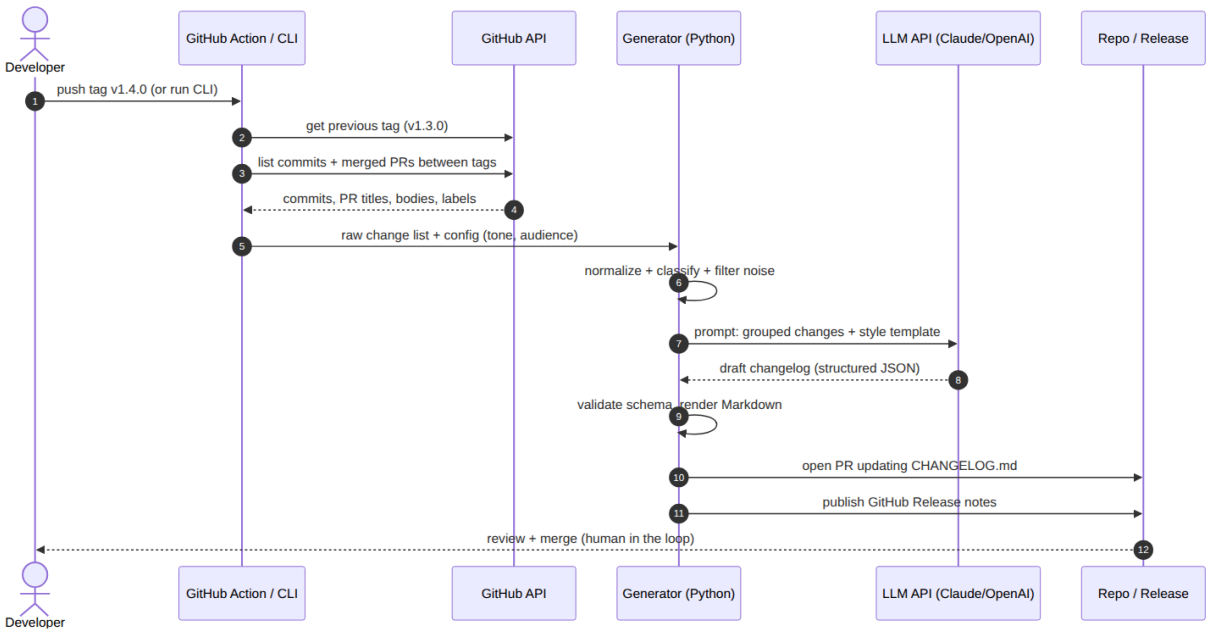

Figure 2 — Sequence: tag push → collect → classify → LLM → validated render → PR for human review.

Key detail: steps 6–9 form the reliability loop. The generator validates the LLM's JSON against a schema; on failure it retries once with the validation error appended; on second failure it falls back to a rule-based render of the grouped changes so the pipeline never blocks a release.

4. Deployment Evolution

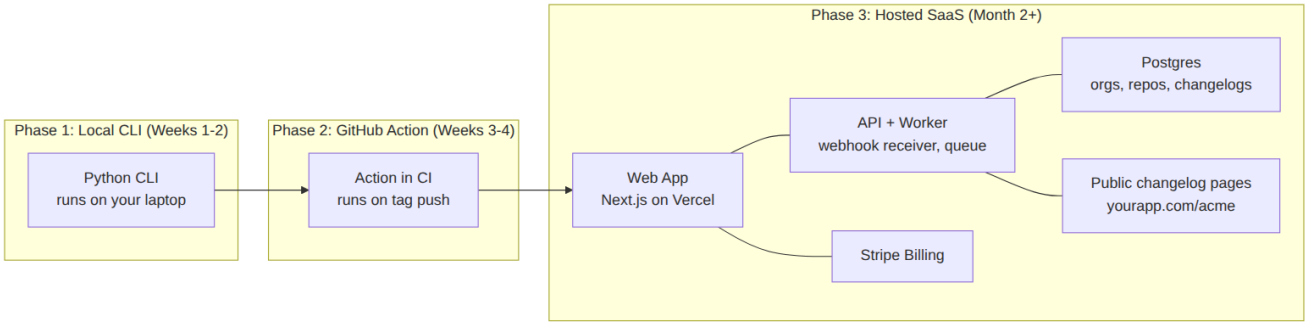

Figure 3 — Same core package deployed three ways as the project matures.

5. Tech Stack

Layer	Choice	Why
Language	Python 3.12	Your daily driver; fastest path; rich SDKs
CLI framework	Typer + Rich	Clean CLI with near-zero boilerplate
LLM	Claude (Haiku/Sonnet tier) or GPT mini-tier, behind an interface	Cheap per release; easy to swap; bring-your-own-key mode later
Prompting	Jinja2 templates + JSON schema outputs	Versionable, testable prompts; no silent format drift
Validation	Pydantic v2	Schema for ChangeItem and LLM response
GitHub integration	REST API via requests; Action = composite action running the CLI	No framework needed at this scale
Packaging	uv + pyproject.toml; publish to PyPI	pipx install changelog-gen
Phase-3 SaaS	Next.js on Vercel + Postgres (Neon/Supabase) + Stripe	Solo-friendly, near-zero ops

6. Data Model & LLM Contract

6.1 ChangeItem (internal model)

```
class ChangeItem(BaseModel):
```

```
    id: str                # sha or PR number
```

```
    source: Literal['commit', 'pr']
```

```
    title: str             # commit subject or PR title
```

```
    body: str | None       # PR body (truncated)
```

```
    labels: list[str]      # PR labels
```

```
    kind: Literal['feature','improvement','fix','breaking','internal']
```

```
    refs: list[str]        # linked issues '#123'
```

6.2 LLM response schema (enforced)

```
{
```

```
  "version": "v1.4.0",
```

```
  "headline": "one-sentence release summary",
```

```
  "sections": [
```

```
    { "name": "Features",
```

```
      "entries": [ { "text": "customer-facing sentence",
```

```
                    "source_ids": ["#482", "a1b2c3"] } ] }
```

```
  ]
```

```
}
```

Every generated entry must cite the source_ids it was derived from — this makes review trivial (the PR diff links prose back to commits) and is your anti-hallucination guard: entries with unknown ids are rejected in validation.

6.3 Prompt design (summary)

- System prompt: role ("you write customer-facing release notes"), tone/audience from config, hard rules (no invented features, one sentence per entry, cite source_ids, skip internal jargon).
- User prompt: product one-liner from config, grouped ChangeItems as compact JSON, 2 few-shot examples of good entries (bad commit → good entry).
- One call per release, not per change — cheaper and lets the model dedupe related commits into one entry.

7. Exact Build Steps — Phase 1 CLI

Repository layout to create (this is the folder pushed to GitHub as ChangeLogScriptGenerator):

```
ChangeLogScriptGenerator/
```

```
├─ pyproject.toml
```

```
├─ README.md
```

```
├─ .changelog.yml            # example config
```

```
├─ action.yml                # Phase 2: composite GitHub Action
```

```
├─ src/changelog_gen/
```

```
│   ├─ __init__.py
```

```
│   ├─ cli.py                # Typer entrypoint
```

```
│   ├─ collect.py            # git + GitHub API
```

```
│   ├─ models.py             # ChangeItem, LLMResponse (Pydantic)
```

```
│   ├─ classify.py           # rules-first bucketing
```

```
│   ├─ llm.py                # provider interface + retry/validate
```

```
│   ├─ prompts/release.j2    # versioned prompt template
```

```
│   └─ render.py             # markdown / release templates
```

```
├─ tests/
```

```
└─ docs/                     # BRD, this design doc, diagrams
```

- **Bootstrap (1 evening).** uv init, add typer, rich, pydantic, requests, jinja2, pyyaml, anthropic. Wire `changelog-gen --help`. Commit, push to GitHub.
- **Collector (1 evening).** `git log prev_tag..new_tag --pretty` via subprocess for commits; GitHub API `/repos/{o}/{r}/pulls?state=closed` filtered to merged-in-range for PRs. Auto-detect previous tag with `git describe --tags --abbrev=0 <tag>^`. Dump raw JSON — verify on a real repo before going further.
- **Normalize + classify (1 evening).** Map conventional prefixes and PR labels to kinds; drop internal kinds; leftovers default to 'improvement'. Unit-test with messy real-world messages.
- **LLM call (1–2 evenings).** Jinja2 prompt → one API call → parse JSON → Pydantic validate (including source_ids exist) → retry once with error feedback → rule-based fallback. This module is where you learn structured outputs properly.
- **Render + ship (1 evening).** Markdown and Release templates; `changelog-gen generate --from v1.3.0 --to v1.4.0 --style friendly`. Run on your repos and 2 friends' repos; iterate on the prompt with real output.
- **Phase 2 — Action (1 weekend).** action.yml (composite): checkout with fetch-depth 0, setup Python, pip install your package, run generate, then peter-evans/create-pull-request to open the

changelog PR. Trigger: push tags 'v*'. Secrets: ANTHROPIC_API_KEY; GITHUB_TOKEN is built in.

Example .changelog.yml

```
product: "Acme API — payments infrastructure for indie SaaS"
```

```
tone: friendly            # professional | friendly | technical
```

```
audience: customers       # customers | developers | internal
```

```
sections: [Features, Improvements, Bug Fixes, Breaking Changes]
```

ignore:

```
  - "chore(*"
```

```
  - "ci:*"
```

```
  - "bump *"
```

```
breaking_labels: [breaking, breaking-change]
```

llm:

```
  provider: anthropic
```

```
  model: claude-haiku     # cheap tier is plenty for this task
```

8. Testing & Quality

- Unit tests: normalizer and classifier on a corpus of ugly real commit messages (collect these from day one — they become your eval set).
- Golden tests: 3 frozen releases with recorded LLM responses (VCR-style) so refactors don't silently change output.

- Prompt evals: a 10-case eval file (input changes → expected qualities checklist: no hallucinated features, correct bucketing, cites sources). Run manually per prompt change in Phase 1; automate with an LLM grader in Phase 2. This is deliberately your on-ramp to LLM evals as a skill.

9. Cost & Operations

- Typical release (≤50 changes) ≈ 3–5K input tokens, ~1K output → well under $0.05 on a cheap-tier model.
- Phase 1–2 has zero infrastructure to operate: it runs in the user's CI with their key. All ops burden is deferred until the SaaS is justified by demand.
